

RECRUITMENT ASSIGNMENT – FRONTEND DEVELOPMENT

Task:

Build a responsive and modern "CodeChef Contest Control Center" frontend.

Problem Statement:

CodeChef VIT is organizing a large-scale competitive programming contest. During the contest, organizers need a centralized dashboard to monitor participants, submissions, rankings, contest status, and overall contest operations.

Your task is to build the frontend for a Contest Control Center that allows organizers to manage and monitor the contest efficiently.

The system should focus on handling dynamic data, real-time-like updates, complex state management, and user interactions rather than simply displaying information.

Tech Stack:

- React.js
- Next.js

Mock data must be used. No backend implementation is required.

Core Features:

1. Contest Overview Dashboard

Display:

- Total Participants
- Total Problems
- Total Submissions
- Accepted Submissions
- Rejected Submissions
- Contest Status (Upcoming / Live / Ended)

Include:

- Statistics Cards
- Recent Activities Feed
- Contest Progress Overview
- Contest Countdown Timer

2. Participant Management

Display all contest participants in a table.

Each participant should contain:

- Name
- Institution
- Current Rank
- Problems Solved
- Penalty Time
- Contest Status

Features:

- Search Participants
- Multi-level Filtering
- Sorting
- Pagination

Filters should work together.

Examples:

- Participants with Rank < 50
- Participants who solved more than 3 problems
- Participants from a selected institution

3. Submission Monitoring System

Display all contest submissions.

Each submission should contain:

- Participant Name
- Problem Name
- Submission Time
- Verdict
- Language Used

Features:

- Filter by Verdict
- Filter by Problem
- Filter by Participant
- Search Functionality

Submission statuses:

- Pending
- Running
- Accepted
- Wrong Answer
- Time Limit Exceeded
- Runtime Error

4. Dynamic Leaderboard

Build a leaderboard displaying:

- Rank
- Participant
- Problems Solved
- Penalty Time

Leaderboard should automatically update when submission data changes.

Ranking must be calculated dynamically based on:

- Number of problems solved
- Penalty Time

5. Contest Freeze Mode

Implement a Leaderboard Freeze feature.

Scenario:

The contest enters Freeze Mode during the final phase.

Requirements:

- New submissions should continue to appear in the Submission Monitoring System.
- Leaderboard rankings should remain unchanged while freeze mode is active.
- Once Freeze Mode is disabled, rankings should be recalculated and updated.

6. Rejudge System

Contest administrators may rejudge submissions.

Example:

A previously accepted solution may become rejected after rejudging.

Requirements:

- Allow verdict changes through the UI.
- Automatically recalculate participant scores.
- Automatically update rankings.
- Reflect all changes across the application.

7. Activity Feed

Display system activities such as:

- New participant joined
- Submission received
- Submission rejudged
- Leaderboard frozen
- Leaderboard unfrozen

Activities should be displayed in reverse chronological order.

Technical Requirements:

- Component-Based Architecture
- Clean Folder Structure
- Reusable Components
- Proper State Management
- Mock Data Layer
- Responsive Design
- Accessibility Considerations

Candidates may use:

- Context API
- Redux Toolkit
- Zustand

Mock Data:

Candidates must create and manage mock datasets for:

- Participants
- Problems
- Submissions
- Contest Activities

The application should function entirely using local/mock data.

Bonus Features:

- Dark / Light Mode
- Drag-and-Drop Widgets
- Export Leaderboard as CSV
- Persist State using Local Storage
- Custom Charts and Analytics
- Keyboard Shortcuts
- Optimistic UI Updates
- Undo Last Rejudge Action

Evaluation Criteria:

- React Fundamentals
- State Management
- Data Modeling
- Component Design
- Code Quality
- Project Structure
- Responsiveness
- Logic Implementation
- Scalability of Solution
- Problem Solving Approach
- Creativity

README MUST CONTAIN:

- Project Setup Instructions
- Tech Stack Used
- Folder Structure Explanation
- State Management Approach
- Data Flow Explanation
- Assumptions Made
- Screenshots
- Deployment Link (if deployed)

GENERAL SUBMISSION GUIDELINES

1. Upload your work to GitHub.
2. Ensure your submission is original.
3. Follow industry-standard best practices.
4. Write clean, maintainable, and well-structured code.
5. Include a README file.
6. Deployment is highly preferred and will be considered a plus during evaluation.

SUBMISSION FORM

We will be sharing the Google Form for submission shortly.

Submission Format:

- Full Name
- Year & Branch
- Track Chosen
- GitHub Repository
- Deployment Link (if applicable)

Important Note:

****Focus on building a scalable and maintainable frontend architecture. We are evaluating how you structure components, manage state, handle dynamic updates, and solve real-world frontend challenges—not just how visually appealing the interface looks. ****

You can take help from AI, but you will be cross examined regarding your logical thinking & creativity. Minimum 70% of the features must be implemented for advancing in the interview round.